Assignment: Voice Spend Manager

Oct 28, 2026 · @Lekhika

Overview

Build a working voice-first spend tracker in about two days of focused work (12–16 hours), and share it within 7 days.

This project shows us how you turn a fuzzy product idea into a shipped system. Speech is messy and money must be exact; the interesting decisions live in the gap between the two.

What we'll look at:

- **Scoping**: what you build, what you cut, and why.
- **Design**: how audio becomes a correct, structured expense.
- **Reliability**: how you know it works, measured rather than asserted.
- **Communication**: how clearly you explain trade-offs and known gaps.

If something here is unclear, make a reasonable assumption, note it in your README, and keep going. Questions are welcome at any point.

The brief

Most people never log their cash and UPI spends, because opening an app and filling a form takes longer than the purchase. Your product fixes that: say what you spent, and it is logged in under 5 seconds.

Design for one user who logs expenses through the day and asks about their spending at the end of the week. Amounts are in rupees unless the user says otherwise.

Things they might say are below. You don't have to handle every one perfectly, but the product must never record a wrong amount silently.

They say	What should happen
"Spent 450 on lunch at Truffles with Rahul"	₹450, Food, merchant Truffles, today, note "with Rahul"

They say	What should happen
"Uber to the airport, 820"	₹820, Transport, merchant Uber
"Paid 2k for electricity yesterday"	₹2,000, Utilities, dated yesterday
"Coffee 180 and parking 50"	Two separate entries
"Spent fifteen hundred"	₹1,500 captured, then it asks what it was for
"Chai aur samosa, saath rupaye"	₹60, Food (Hinglish is a stretch goal)
"Actually, make that 500"	Updates the entry just logged
"Delete the last one"	Deletes it after a confirmation
"How much did I spend on food this week?"	A total computed from saved entries
"Am I over budget on shopping?"	This month's shopping spend against its budget

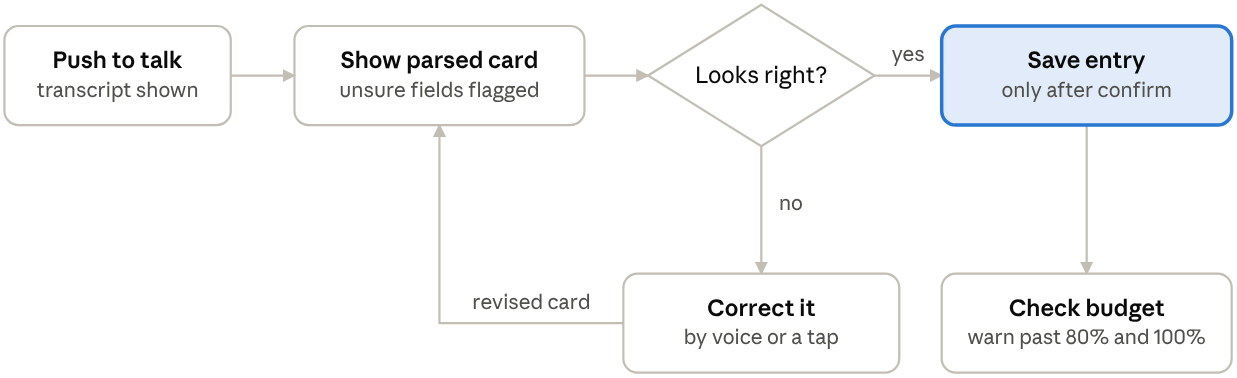
Core requirements

All eight are required and listed in priority order. A smaller set that works reliably beats all eight half-done, so if you run short, cut from the bottom and tell us what you cut.

1. **Voice capture.** Push-to-talk in a web or mobile app, with the transcript shown as the user speaks or right after.
2. **Parsing.** Turn an utterance into amount, currency, category, merchant, date and note. Handle number words ("fifteen hundred"), shorthand ("2k", "1.5 lakh") and relative dates ("yesterday", "last Friday").
3. **Confirm before saving.** Show the parsed expense as a card. The user confirms, or corrects it by voice ("make it 500") or by tapping a field. Flag any field the system is unsure of.
4. **Persistence.** A real datastore, so data survives a restart. One user, no login needed.
5. **History.** A plain list view to browse, edit and delete entries without speaking.
6. **Questions by voice.** Answer questions about spending: totals by category or time range, top merchants, recent entries. Every number in an answer comes from saved data, never from a model's guess.
7. **Budgets.** Set a monthly budget per category, by voice or in the UI. Warn when an entry takes a category past 80% and past 100% of its budget.

8. **Several expenses in one utterance.** "Coffee 180 and parking 50" becomes two entries.

Nothing is saved until the user confirms the card



logging loop · 5 steps, 1 decision

A wrong or unsure field loops back through a correction; only a confirmed card becomes an entry.

Use these categories, or justify a different set: Food, Groceries, Transport, Shopping, Utilities, Entertainment, Health, Travel, Education, Other.

Stretch goals

Optional. Pick at most two and do them well; we value depth over breadth, and skipping them all is fine.

- **Hinglish and code-switched speech**, with accuracy measured on your eval set.
- **Spoken replies** (text-to-speech), for a fully hands-free loop.
- **Splits**: "Dinner was 3,600, I paid, split four ways" records your share and what each friend owes.
- **Recurring expenses**: "Netflix 649 every month" logs itself each month.
- **Receipt photo plus voice**: a photo of a bill, with a voice note adding context.
- **Offline capture**: record without a network, parse when back online.
- **Latency**: under 1.5 seconds from end of speech to the confirmation card, with measurements to back it.
- **Weekly summary**: a short spoken or written recap of the week's spending.

Constraints and guidance

- **Stack**: your choice. A web app is enough; native mobile is not required.
- **Speech and models**: use any speech-to-text service and any LLM. Tell us why you picked them and roughly what they cost per 100 expenses logged.
- **AI coding tools**: allowed and expected. Tell us briefly how you used them; you should be able to explain every line in the review.
- **Money**: amounts must be exact. Tell us how you store and round them.
- **Privacy**: voice is personal data. State where audio goes, whether you keep it, and for how long.
- **Skip**: login, multiple users, polished visual design and production deployment. A hosted demo link is a plus, not a requirement.
- **API costs**: if paid APIs are a blocker, tell us before you start.

Deliverables

Send one link: a Git repo, or a zip that includes the `.git` folder.

Deliverable	What it should contain
Code	The full repo with its real commit history, not one squashed commit
README	How to run it in 3 commands or fewer, the env vars it needs, and known gaps
Design note	1–2 pages: an architecture diagram, how an utterance becomes a saved expense, how you prevent wrong amounts, your main trade-offs, and what you'd do with another week
Eval set	At least 30 labelled utterances with expected output, plus a script that runs them and reports accuracy per field
Tests	Unit tests for amount and date parsing, at minimum
Demo video	3–5 minutes showing logging, a correction, a question and a budget warning

How we'll review it

Correctness on messy input counts most; visual polish counts least.

Area	Weight	What good looks like
Parsing quality and safety	30%	High per-field accuracy on your eval set and on ours; ambiguous input gets a question, not a guess; no amount is saved without confirmation
Works end to end	25%	Runs from the README; the core flows work when we try them ourselves
Design and code	20%	Clear boundaries between speech, parsing, storage and queries; exact money handling; tests where the risk is
Product judgment	15%	Corrections feel quick; sensible cuts; latency you measured and thought about
Communication	10%	A design note we can follow; honest about gaps and what comes next

The review session

After you submit, we'll book a 60-minute call to go through it together.

1. **Live demo, 10 min.** You run it; we say a few things that aren't in your eval set.
2. **Design walkthrough, 20 min.** Your architecture, and what you'd change to serve 10,000 users.
3. **Live change, 20 min.** We pick a small extension, such as a new category or a new date phrase, and build it together.
4. **Your questions, 10 min.**